

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5  #include <sys/types.h>
6
7  void create_children_wait()
8  {
9      int i;
10     pid_t pid;
11     int status;
12
13     printf("\n===== USING wait() =====\n");
14
15     for (i = 0; i < 3; i++)
16     {
17         pid = fork();
18
19         if (pid < 0)
20         {
21             perror("fork failed");
22             exit(1);
23         }
24
25         if (pid == 0)
26         {
27             printf("Child %d: PID = %d, Parent PID = %d\n",
28                   i + 1, getpid(), getppid());
29
30             sleep(2 + i);
31
32             printf("Child %d: Finished\n", i + 1);
33             exit(10 + i);
34         }
35     }
36
37     /* Parent waits for any child */
38     for (i = 0; i < 3; i++)
39     {
40         pid = wait(&status);
41
42         if (pid > 0)
43         {
44             if (WIFEXITED(status))
45             {
46                 printf("Parent: Child PID %d completed with status %d\n",
47                       pid, WEXITSTATUS(status));
48             }
49         }
50     }
51
52     printf("Parent: All children completed using wait().\n");
53 }
54
55 void create_children_waitpid()
```

```

56 {
57     int i;
58     pid_t pid;
59     pid_t child_pids[3];
60     int status;
61
62     printf("\n===== USING waitpid() =====\n");
63
64     for (i = 0; i < 3; i++)
65     {
66         pid = fork();
67
68         if (pid < 0)
69         {
70             perror("fork failed");
71             exit(1);
72         }
73
74         if (pid == 0)
75         {
76             printf("Child %d: PID = %d, Parent PID = %d\n",
77                   i + 1, getpid(), getppid());
78             sleep(2 + i);
79             printf("Child %d: Finished\n", i + 1);
80             exit(20 + i);
81         }
82         child_pids[i] = pid;
83     }
84
85     /*
86     * Parent waits for a specific child
87     * using waitpid().
88     */
89     for (i = 0; i < 3; i++)
90     {
91         pid = waitpid(child_pids[i], &status, 0);
92
93         if (pid > 0)
94         {
95             if (WIFEXITED(status))
96             {
97                 printf("Parent: Specific child PID %d completed "
98                       "with status %d\n",
99                       pid, WEXITSTATUS(status));
100             }
101         }
102     }
103
104     printf("Parent: All children completed using waitpid().\n");
105 }
106
107 void create_zombie()

```

```

111 {
112     pid_t pid;
113
114     printf("\n==== ZOMBIE PROCESS DEMONSTRATION =====\n");
115
116     pid = fork();
117
118     if (pid < 0)
119     {
120         perror("Fork failed");
121         exit(1);
122     }
123
124     if (pid == 0)
125     {
126         printf("Zombie child: PID = %d\n", getpid());
127         printf("Zombie child: Exiting immediately...\n");
128
129         exit(0);
130     }
131     else
132     {
133         printf("Parent: PID = %d\n", getpid());
134         printf("Parent: Child PID = %d\n", pid);
135
136         printf("Parent: Sleeping for 10 seconds.\n");
137         printf("During this time, the child becomes a ZOMBIE.\n");
138
139         sleep(10);
140
141         printf("Parent: Now calling wait() to remove zombie.\n");
142
143         wait(NULL);
144
145         printf("Parent: Zombie process has been cleaned up.\n");
146     }
147 }
148
149 void eliminate_zombie()
150 {
151     pid_t pid;
152     int status;
153
154     printf("\n==== ELIMINATING ZOMBIE USING waitpid() =====\n");
155
156     pid = fork();
157
158     if (pid < 0)
159     {
160         perror("Fork failed");
161         exit(1);
162     }
163
164     if (pid == 0)
165     {

```

```

166     printf("Child: PID = %d\n", getpid());
167     printf("Child: Doing some work...\n");
168
169     sleep(2);
170
171     printf("Child: Exiting...\n");
172     exit(0);
173 }
174 else
175 {
176     printf("Parent: PID = %d\n", getpid());
177     printf("Parent: Waiting for child PID %d using waitpid().\n",
178           pid);
179
180     waitpid(pid, &status, 0);
181
182     if (WIFEXITED(status))
183     {
184         printf("Parent: Child terminated normally.\n");
185         printf("Parent: Exit status = %d\n",
186               WEXITSTATUS(status));
187     }
188
189     printf("Parent: No zombie process remains.\n");
190 }
191 }
192
193 int main()
194 {
195     printf("=====\n");
196     printf(" PROCESS SYNCHRONIZATION DEMONSTRATION\n");
197     printf("=====\n");
198
199     /* Part 1: wait() */
200     create_children_wait();
201
202     /* Part 2: waitpid() */
203     create_children_waitpid();
204
205     /* Part 3: Create zombie */
206     create_zombie();
207
208     /* Part 4: Remove zombie properly */
209     eliminate_zombie();
210
211     printf("\n=====\n");
212     printf("Program completed successfully.\n");
213     printf("=====\n");
214
215     return 0;
216 }
217

```

Output:

```
saisurendra@SaiSurendra:~$ nano Practical17.c
saisurendra@SaiSurendra:~$ gcc Practical17.c -o Practical17
saisurendra@SaiSurendra:~$ ./Practical17
```

```
=====
PROCESS SYNCHRONIZATION DEMONSTRATION
=====
```

```
===== USING wait() =====
```

```
Child 1: PID = 898, Parent PID = 897
Child 2: PID = 899, Parent PID = 897
Child 3: PID = 900, Parent PID = 897
Child 1: Finished
Parent: Child PID 898 completed with status 10
Child 2: Finished
Parent: Child PID 899 completed with status 11
Child 3: Finished
Parent: Child PID 900 completed with status 12
Parent: All children completed using wait().
```

```
===== USING waitpid() =====
```

```
Child 1: PID = 901, Parent PID = 897
Child 3: PID = 903, Parent PID = 897
Child 2: PID = 902, Parent PID = 897
Child 1: Finished
Parent: Specific child PID 901 completed with status 20
Child 2: Finished
Parent: Specific child PID 902 completed with status 21
Child 3: Finished
Parent: Specific child PID 903 completed with status 22
Parent: All children completed using waitpid().
```

```
===== ZOMBIE PROCESS DEMONSTRATION =====
```

```
Parent: PID = 897
Zombie child: PID = 904
Parent: Child PID = 904
Zombie child: Exiting immediately...
Parent: Sleeping for 10 seconds.
During this time, the child becomes a ZOMBIE.
Parent: Now calling wait() to remove zombie.
Parent: Zombie process has been cleaned up.
```

```
===== ELIMINATING ZOMBIE USING waitpid() =====
```

```
Parent: PID = 897
Parent: Waiting for child PID 905 using waitpid().
Child: PID = 905
Child: Doing some work...
Child: Exiting...
Parent: Child terminated normally.
Parent: Exit status = 0
Parent: No zombie process remains.
```

```
=====
Program completed successfully.
=====
```